

POWERTRACK – CONSTRUYENDO UN SISTEMA

Ciclo de Vida del Desarrollo de
Software (SDLC)



¿QUÉ ES EL SDLC?



Definición: Software Development Life Cycle (Ciclo de Vida del Desarrollo de Software) — un marco metodológico que guía todo el proceso de creación de software.

Objetivo: Proceso estructurado para planificar, crear, probar y entregar software de calidad, minimizando riesgos, costos y errores en cada etapa.

LAS 7 FASES PRINCIPALES DEL SDLC

- 1. Planificación
- 1. Análisis de requisitos
- 1. Diseño del sistema
- 1. Implementación (codificación)
- 1. Pruebas y calidad
- 1. Despliegue
- 1. Mantenimiento



IMPORTANCIA DEL ALCANCE

Scope Creep: expansión descontrolada del alcance del proyecto sin ajustes en tiempo, presupuesto o recursos. Ocurre cuando se agregan funcionalidades no planificadas.

Ejemplo POWERTRACK: el sistema gestiona únicamente tres módulos definidos: usuarios, pagos y rutinas. Cualquier adición fuera de este alcance requiere evaluación formal.

Riesgos de no definir el alcance: pérdida de tiempo valioso, presupuesto excedido significativamente, retrasos en la entrega y frustración del equipo.



CLASIFICACIÓN DE REQUISITOS

Requisitos Funcionales: Describen las acciones concretas que el sistema debe realizar. Definen el "qué hace" el software — funcionalidades específicas y comportamientos esperados.

DOS TIPOS FUNDAMENTALES DE REQUISITOS

Requisitos No Funcionales: Describen las cualidades o características del sistema como rendimiento, escalabilidad, seguridad y usabilidad. Definen el "cómo lo hace" el software.





POWERTRACK

EJEMPLOS DE REQUISITOS

REQUISITOS FUNCIONALES

- Iniciar sesión con correo electrónico y contraseña
- Registrar pagos mensuales de membresía
 - Gestionar rutinas de entrenamiento

REQUISITOS NO FUNCIONALES

- El sistema debe responder en menos de 2 segundos
 - Debe soportar 500 usuarios simultáneos
- Alta disponibilidad 99.9% del tiempo

COMPARACIÓN CLAVE

Los funcionales describen QUÉ hace el sistema (acciones concretas), mientras los no funcionales describen CÓMO lo hace (calidades de rendimiento y escalabilidad).



POWERTRACK

DISEÑO DEL SISTEMA

HLD — DISEÑO DE ALTO NIVEL

Plano general del sistema.
Estructura y comunicación
entre módulos. Ejemplo:
usuarios, pagos, rutinas.
Visión arquitectónica global.

LLD — DISEÑO DE BAJO NIVEL

Detalles técnicos
específicos. Define
funciones, métodos y
algoritmos. Especifica
tablas, campos y relaciones
de BD. Orientado a
programadores.

COMPARACIÓN VISUAL

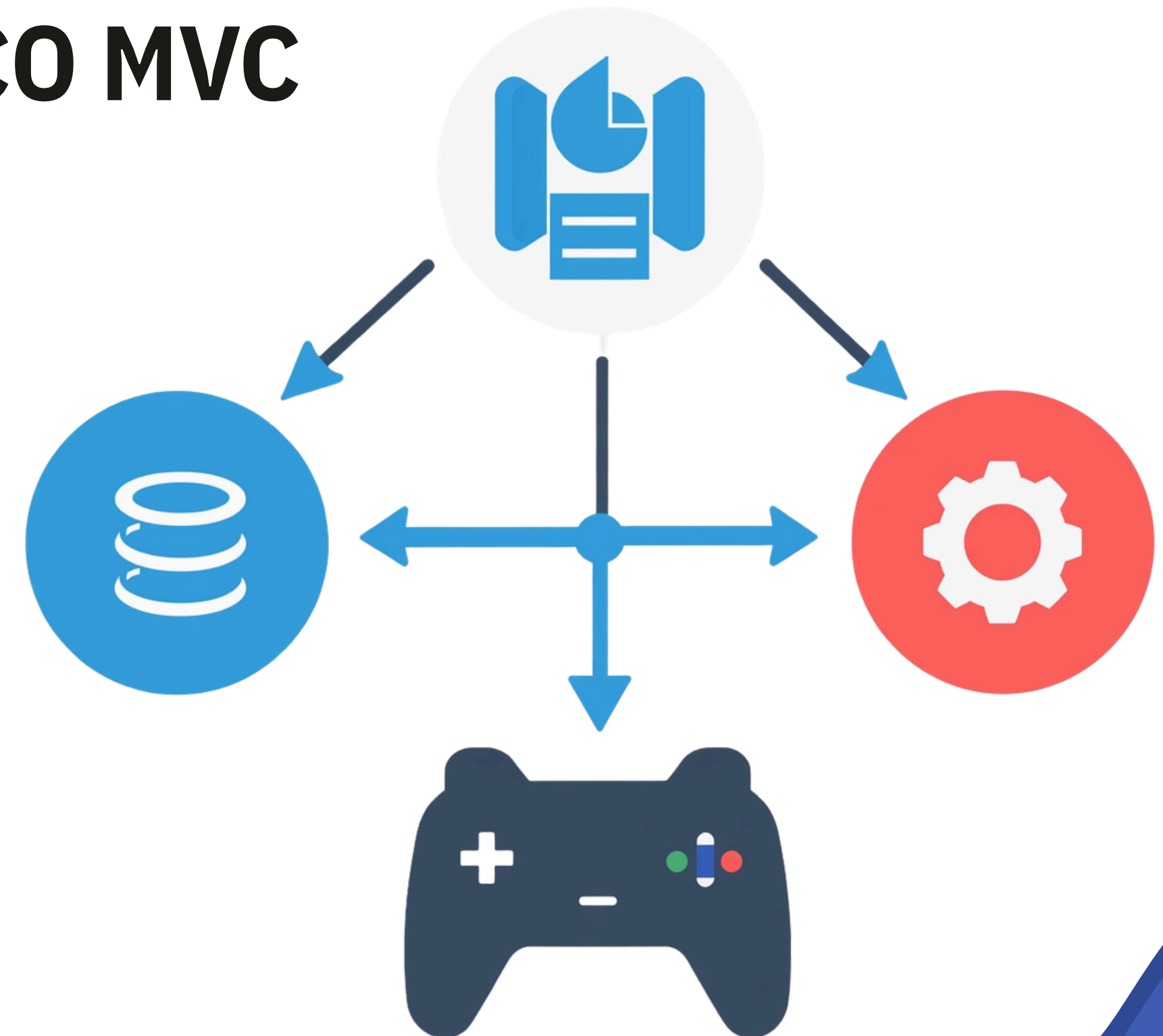
HLD define QUÉ hace el
sistema. LLD define CÓMO
lo hace. Ambos son
esenciales para un
desarrollo exitoso.

PATRÓN ARQUITECTÓNICO MVC

Modelo: Gestiona los datos y la lógica de negocio. Contiene las reglas del sistema y se comunica con la base de datos.

Vista: Es la interfaz de usuario. Presenta la información al usuario y captura sus interacciones con el sistema.

Controlador: Actúa como intermediario entre el Modelo y la Vista. Procesa las solicitudes del usuario y coordina las respuestas.



IMPLEMENTACIÓN – ¿QUÉ ES CLEAN CODE?

Código fácil de leer y mantener

Nombres descriptivos para variables y funciones

Funciones cortas y concisas

Responsabilidad única por función

Evitar comentarios innecesarios

No repetir lógica (DRY)



EJEMPLO DE CLEAN CODE EN POWERTRACK

Función con nombre claro y corta:
`calcularFechaVencimiento(fechaRegistro)`
Devuelve fecha exacta un mes después, sin lógica
duplicada ni código innecesario.

APLICACIÓN DE PRINCIPIOS CONCRETOS

- Código legible sin repeticiones:
- Nombres descriptivos que explican su propósito
 - Una sola responsabilidad por función
 - Sin comentarios redundantes



PRUEBAS Y CALIDAD

Validan funciones o métodos individuales de forma aislada, verificando que cada componente funcione correctamente por separado.

Detectan errores temprano en el ciclo de desarrollo, reduciendo costos de corrección y evitando problemas mayores en producción.

Aseguran la calidad del software al proporcionar confianza en que el código cumple con los requisitos esperados.



EJEMPLO DE PRUEBA UNITARIA EN POWERTRACK



Qué hace: Recibe fecha de registro y devuelve la fecha de vencimiento (un mes después).
Ejemplo: registro 15/01 → vencimiento 15/02.

FUNCIÓN: `CALCULARFECHAVENCIMIENTO()`

Test: Validamos que el resultado sea exactamente el esperado. Beneficio: confianza en funcionalidad crítica del sistema de pagos.

DESPLIEGUE Y MANTENIMIENTO – CI/CD

CI (Integración Continua): Integración frecuente del código con pruebas automáticas que detectan errores de forma temprana, antes de que lleguen a producción.

BENEFICIOS CLAVE DE CI/CD

CD (Entrega/Despliegue Continuo): Automatización del proceso para llevar código a producción rápidamente. Reduce errores manuales y tiempos de entrega significativamente.



DOCKER — CONTENEDORES PARA APLICACIONES

Docker empaqueta tu aplicación junto con todas sus dependencias en un contenedor portátil. Esto garantiza que la aplicación se ejecute de manera idéntica en cualquier entorno: desarrollo, pruebas o producción. Facilita enormemente el despliegue y mantenimiento del software.





POWERTRACK

TIPOS DE MANTENIMIENTO

MANTENIMIENTO CORRECTIVO

Corregir errores existentes en el sistema. Ejemplo POWERTRACK: solucionar bug en el registro de pagos que duplicaba transacciones.

MANTENIMIENTO EVOLUTIVO

Agregar o mejorar funcionalidades. Ejemplo POWERTRACK: implementar nuevo módulo de seguimiento de progreso físico para los usuarios.

DIFERENCIAS CLAVE

Correctivo reacciona a problemas existentes.
Evolutivo anticipa necesidades futuras. Ambos son esenciales para la salud del software a largo plazo.



POWERTRACK

ANÁLISIS DE CASO REAL

CASO: APLICACIÓN BANCARIA CON FALLA DE USUARIOS CONCURRENTES

Una aplicación bancaria experimentó caídas críticas cuando múltiples usuarios intentaron acceder simultáneamente. El sistema no fue diseñado para manejar la carga real de producción.

REQUISITO QUE FALLÓ:

Requisito no funcional de escalabilidad y rendimiento. El sistema no soportaba la cantidad de usuarios simultáneos necesarios para operación normal.

FASES DE DETECCIÓN:

- Análisis de requisitos: definir cantidad de usuarios concurrentes esperados
- Pruebas: realizar load testing y stress testing para validar capacidad del sistema

LECCIÓN CLAVE:

Detectar fallas temprano evita costos y riesgos graves, especialmente en sistemas críticos como aplicaciones bancarias donde la disponibilidad es esencial.